Enrollment no-240840116014

Practical - 04

AIM:Perform Exploratory Data Analysis (EDA) using descriptive statistics and identify trends in datasets.

Step 1: import Libraries

```
# Import the required libraries import pandas as pd
import pandas as pd

import  matplotlib.pyplot  as  plt
```

Step 2: Create a Dataset

```
#Create a dictionary containing monthly sales data such as Month, Units Sold, Rever data
data = {
    'Month': ['January', 'February', 'March', 'April', 'May', 'June',
              'July', 'August', 'September', 'October', 'November', 'December'],

    'Units_Sold': [120, 135, 150, 145, 160, 175,
                   180, 170, 190, 210, 230, 250],

    'Revenue': [120000, 135000, 150000, 145000, 160000, 175000,
                180000, 170000, 190000, 210000, 230000, 250000],

    'Profit': [20000, 23000, 25000, 24000, 27000, 30000,
               31000, 29000, 33000, 37000, 41000, 45000]
}

df = pd.DataFrame(data)

print(df)
```

```
        Month  Units_Sold  Revenue  Profit
0     January         120   120000   20000
1    February         135   135000   23000
2       March         150   150000   25000
3       April         145   145000   24000
4         May         160   160000   27000
5        June         175   175000   30000
6        July         180   180000   31000
7      August         170   170000   29000
8   September         190   190000   33000
9     October         210   210000   37000
10   November         230   230000   41000
11   December         250   250000   45000
```

Step 3: Inspect the **dataset**

```
# Display the first 5 rows of the dataset
df.head()
```

	Month	Units_Sold	Revenue	Profit
0	January	120	120000	20000
1	February	135	135000	23000
2	March	150	150000	25000
3	April	145	145000	24000
4	May	160	160000	27000

```
# Display the number of rows and columns
df.shape
```

```
(12, 4)
```

```
# Display columns about the dataset
df.columns
```

```
Index(['Month', 'Units_Sold', 'Revenue', 'Profit'], dtype='object')
```

```
# Display detailed information
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 12 entries, 0 to 11
Data columns (total 4 columns):
 #   Column      Non-Null Count  Dtype 
---  ------      --------------  ----- 
 0   Month       12 non-null     object
 1   Units_Sold  12 non-null     int64 
 2   Revenue     12 non-null     int64 
 3   Profit      12 non-null     int64 
dtypes: int64(3), object(1)
memory usage: 516.0+ bytes
```

Step 4: Generate discriptive statistic

```
# Calculate statistical measures
df.describe()
```

	Units_Sold	Revenue	Profit
count	12.000000	12.000000	12.000000
mean	176.250000	176250.000000	30416.666667
std	38.737168	38737.167963	7525.210155
min	120.000000	120000.000000	20000.000000
25%	148.750000	148750.000000	24750.000000
50%	172.500000	172500.000000	29500.000000
75%	195.000000	195000.000000	34000.000000
max	250.000000	250000.000000	45000.000000

Step 5: Calculate mean

```
# Calculate the average number of units sold during the year
average_units = df['Units_Sold'].mean()
print("Average Units Sold:", average_units)
```

```
Average Units Sold: 176.25
```

Step 6: Calculate median

```
# Find the middle value of the Units_Sold data
median_units = df['Units_Sold'].median()
print("Median Units Sold:", median_units)
```

```
Median Units Sold: 172.5
```

Step: 7 Calculate max and min

```
# Find the highest and lowest number of units sold
maximum_units = df['Units_Sold'].max()
minimum_units = df['Units_Sold'].min()
print("Maximum Units Sold:", maximum_units)
print("Minimum Units Sold:", minimum_units)
```

```
Maximum Units Sold: 250
Minimum Units Sold: 120
```

Step 8: Calculate standard deviation

```
# Calculate the standard deviation of Units_Sold to measure the variation in month]
std_units = df['Units_Sold'].std()
print("Standard Deviation of Units Sold:", std_units)
```

```
Standard Deviation of Units Sold: 38.73716796331634
```

Step 9: Find the month with highest sales

```
# find the index of the row having the maximum revenue
highest_sales_index = df['Revenue'].idxmax()
highest_sales_month = df.loc[highest_sales_index, 'Month']
highest_revenue =   df.loc[highest_sales_index, 'Revenue']
print("Month with Highest Sales:", highest_sales_month)
print("Highest Revenue:", highest_revenue)
```

```
Month with Highest Sales: December
Highest Revenue: 250000
```

Step 10: Find the month with lowest sales

```
# Find the index of the row having the minimum revenue
lowest_sales_index = df['Revenue'].idxmin()
lowest_sales_month  = df.loc[lowest_sales_index, 'Month']
lowest_revenue = df.loc[lowest_sales_index, 'Revenue']
print("Month with Lowest Sales:", lowest_sales_month)
print("Lowest Revenue:", lowest_revenue)
```

```
Month with Lowest Sales: January
Lowest Revenue: 120000
```

Step 11: Call total reven

```
# Calculate the total revenue generated during all 12 months.
total_revenue = df['Revenue'].sum()
print("Total Revenue:", total_revenue)
```

```
Total Revenue: 2115000
```

Step 12: Call total profit

```
# Calculate the total profit earned during all 12 months.
total_profit = df["Profit"].sum()
print("Total Profit:", total_profit)
```

```
Total Profit: 365000
```

Step 13: Identify the sales

```
import matplotlib.pyplot as plt

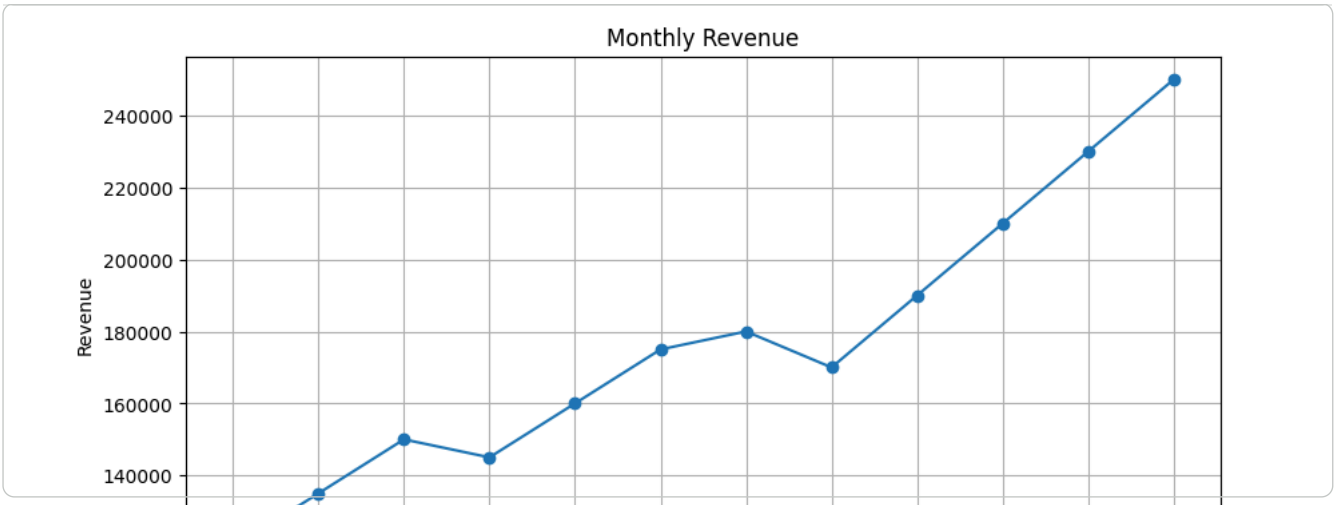
plt.figure(figsize=(10, 5))

plt.plot(df['Month'], df['Revenue'], marker='o')

plt.xlabel('Month')
plt.ylabel('Revenue')
plt.title('Monthly Revenue')

plt.xticks(rotation=45)
plt.grid(True)

plt.show()
```



Step 14: Analyze profit

```
#create a line chart for monthly profit
import matplotlib.pyplot as plt

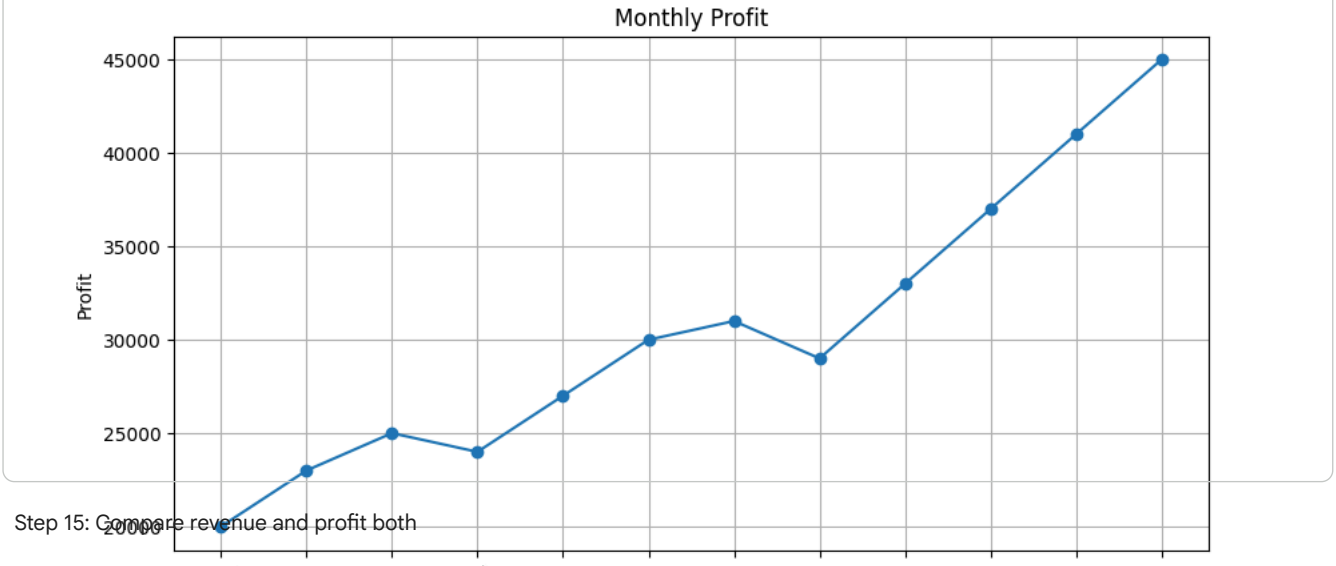
plt.figure(figsize=(10, 5))

plt.plot(df['Month'], df['Profit'], marker='o')

plt.xlabel('Month')
plt.ylabel('Profit')
plt.title('Monthly Profit')

plt.xticks(rotation=45)
plt.grid(True)

plt.show()
```



Step 15: Compare revenue and profit both

```
import matplotlib.pyplot as plt

# Create a line chart containing both Revenue and Profit
plt.figure(figsize=(10, 5))

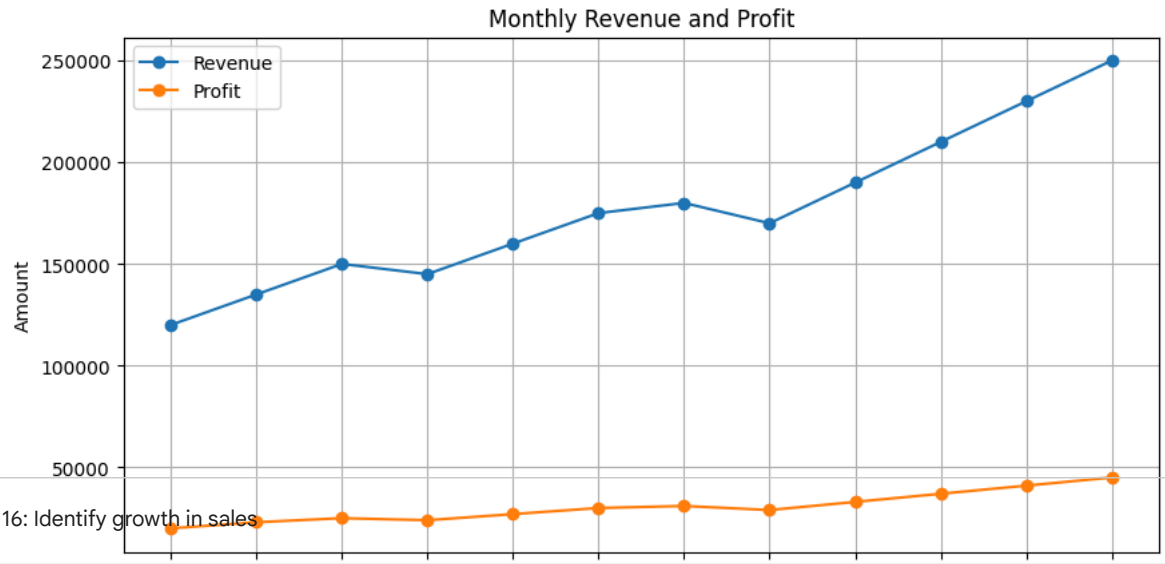
plt.plot(df['Month'], df['Revenue'], marker='o', label='Revenue')
plt.plot(df['Month'], df['Profit'], marker='o', label='Profit')

plt.xlabel('Month')
plt.ylabel('Amount')
plt.title('Monthly Revenue and Profit')

plt.xticks(rotation=45)

plt.legend()
plt.grid(True)

plt.show()
```



Step 16: Identify growth in sales

```
# Calculate the percentage change in revenue from one month to the next month
df['Revenue_Growth_%']  =  df['Revenue'].pct_change()  *  100
df
```

	Month	Units_Sold	Revenue	Profit	Revenue_Growth_%
0	January	120	120000	20000	NaN
1	February	135	135000	23000	12.500000
2	March	150	150000	25000	11.111111
3	April	145	145000	24000	-3.333333
4	May	160	160000	27000	10.344828
5	June	175	175000	30000	9.375000
6	July	180	180000	31000	2.857143
7	August	170	170000	29000	-5.555556
8	September	190	190000	33000	11.764706
9	October	210	210000	37000	10.526316
10	November	230	230000	41000	9.523810
11	December	250	250000	45000	8.695652

Step 17: Identify the months with increasing revenue

```
# Filter the dataset to find the months where revenue growth is greater than 0%
increasing_sales = df[df['Revenue_Growth_%'] > 0]
df
```

	Month	Units_Sold	Revenue	Profit	Revenue_Growth_%
0	January	120	120000	20000	NaN
1	February	135	135000	23000	12.500000
2	March	150	150000	25000	11.111111
3	April	145	145000	24000	-3.333333
4	May	160	160000	27000	10.344828
5	June	175	175000	30000	9.375000
6	July	180	180000	31000	2.857143
7	August	170	170000	29000	-5.555556
8	September	190	190000	33000	11.764706
9	October	210	210000	37000	10.526316
10	November	230	230000	41000	9.523810
11	December	250	250000	45000	8.695652